Xi'an Jiaotong-Liverpool University
西交利物浦大學

XJTLU Entrepreneur College (Taicang) Cover Sheet

Module code and Title	DTS106TC: Introduction to Database	
School Title	School of AI and Advanced Computing	
Assignment Title	Assessment Task 002 (CW)): Individual Coursework	
Submission Deadline	**May 29th ,2026 at 11:59 PM**	
Final Word Count	NA	
If you agree to let the university use your work anonymously for teaching and learning purposes, please type **"yes"** here.		**Yes**

I certify that I have read and understood the University's Policy for dealing with Plagiarism, Collusion and the Fabrication of Data (available on Learning Mall Online). With reference to this policy I certify that:

- My work does not contain any instances of plagiarism and/or collusion.
  My work does not contain any fabricated data.

By uploading my assignment onto Learning Mall Online, I formally declare that all of the above information is true to the best of my knowledge and belief.

Scoring – For Tutor Use	
Student ID	**2468490**

Stage of Marking	**Marker Code**	**Learning Outcomes Achieved (F/P/M/D) (please modify as appropriate)**					**Final Score**
		A	**B**	**C**	**D**	**E**	
1st Marker – red pen							
Moderation – green pen	**IM Initials**	The original mark has been accepted by the moderator (please circle as appropriate):					Y / N
		Data entry and score calculation have been checked by another tutor (please circle):					Y
2nd Marker if needed – green pen							

For Academic Office Use			**Possible Academic Infringement (please tick as appropriate)**	
Date Received	**Days late**	**Late Penalty**	☐ **Category A**	Total Academic Infringement Penalty (A,B, C, D, E, Please modify where necessary)
			☐ **Category B**	_____
			☐ **Category C**	
			☐ **Category D**	
			☐ **Category E**	

Xi'an Jiaotong-Liverpool University
西交利物浦大学

QUESTION 1

ER Diagram:

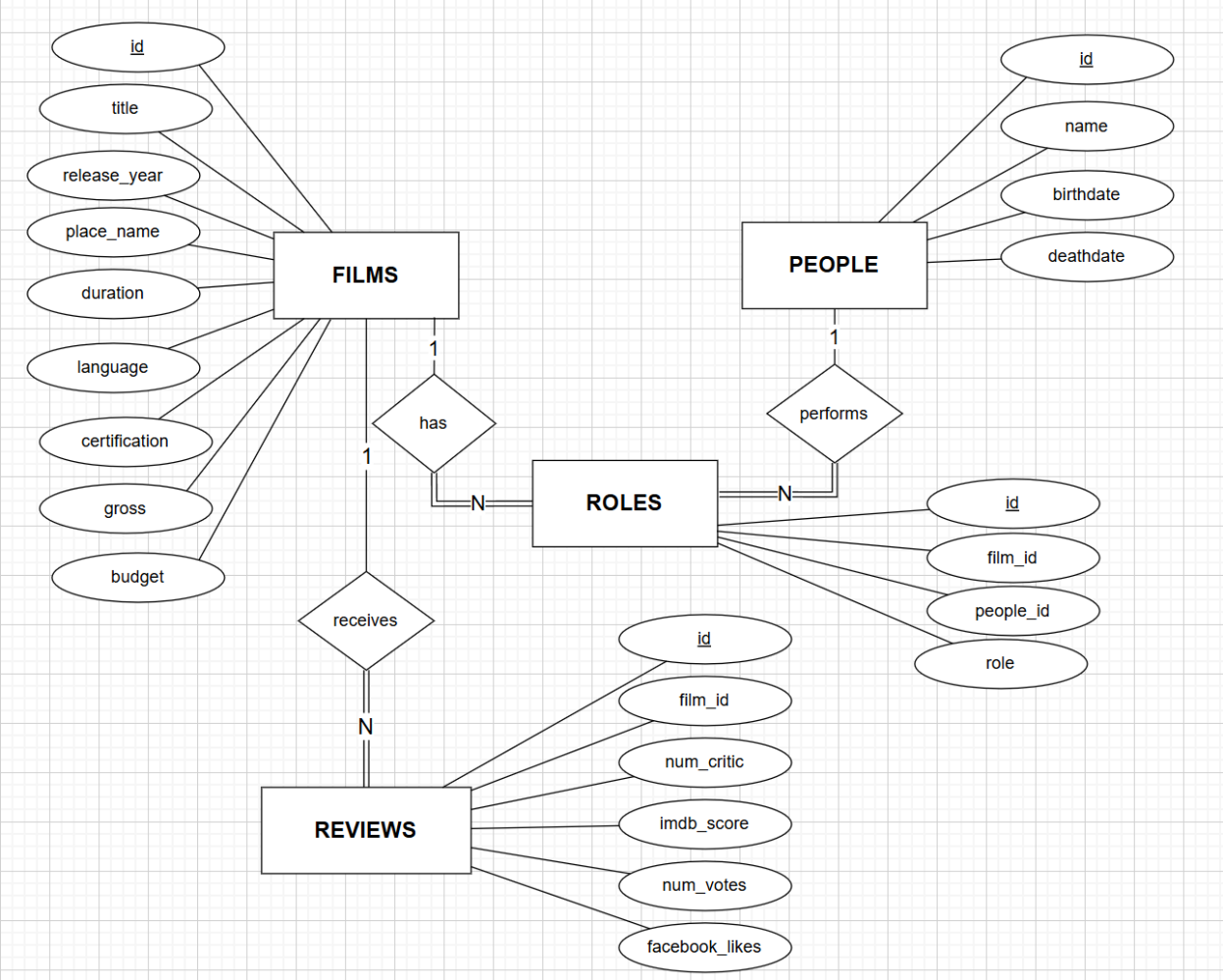


Figure: ER Diagram for Film Database

Constraints and Assumptions

The id attribute in each entity is used as the **primary key**, and it is underlined in the ER diagram. The foreign key reviews.film_id references films.id, while roles.film_id references

films.id and roles.people_id references people.id. These foreign keys maintain referential integrity between each entities.

Participation constraints are considered in the ER diagram. **REVIEWS** must be linked to one FILM, so it has total participation. **ROLES** must be linked to one **FILM** and one **PEOPLE**, so it also has total participation. **FILMS** and **PEOPLE** have partial participation because they may exist without related review or role records.

Attribute constraints are also assumed. For example: 1.title and name should not be null 2. imdb_score should be between 0 and 10 3. numerical values (such as budget, gross, num_votes, num_critic, and facebook_likes) should not be negative.

ER Diagram Explanation

The ER diagram was designed based on the four CSV files: **films**, **people**, **reviews**, and **roles**. Four main entities were identified: **FILMS**, **PEOPLE**, **REVIEWS**, and **ROLES**.

The **FILMS** entity stores basic film information, such as title, release_year, duration, language, certification, gross, and budget. The **PEOPLE** entity stores information about people involved in films, including name, birthdate, and deathdate. The **REVIEWS** entity stores review-related data, including imdb_score, num_votes, num_critic, and facebook_likes. The **ROLES** entity records the role of a person in a specific film.

In my design, **FILMS** has a one-to-many relationship with **REVIEWS**, because one film can receive many review records, while each review belongs to one film. **FILMS** also has a one-to-many relationship with **ROLES**, because one film can have many role records. **PEOPLE** has a one-to-many relationship with **ROLES**, because one person can perform roles in multiple films. Actually, there is a many-to-many relationship between **FILMS** and **PEOPLE**, but it is resolved by the **ROLES** associative entity.

QUESTION 2

Based on the ER diagram in Q1, the conceptual model was converted into the following relational model:

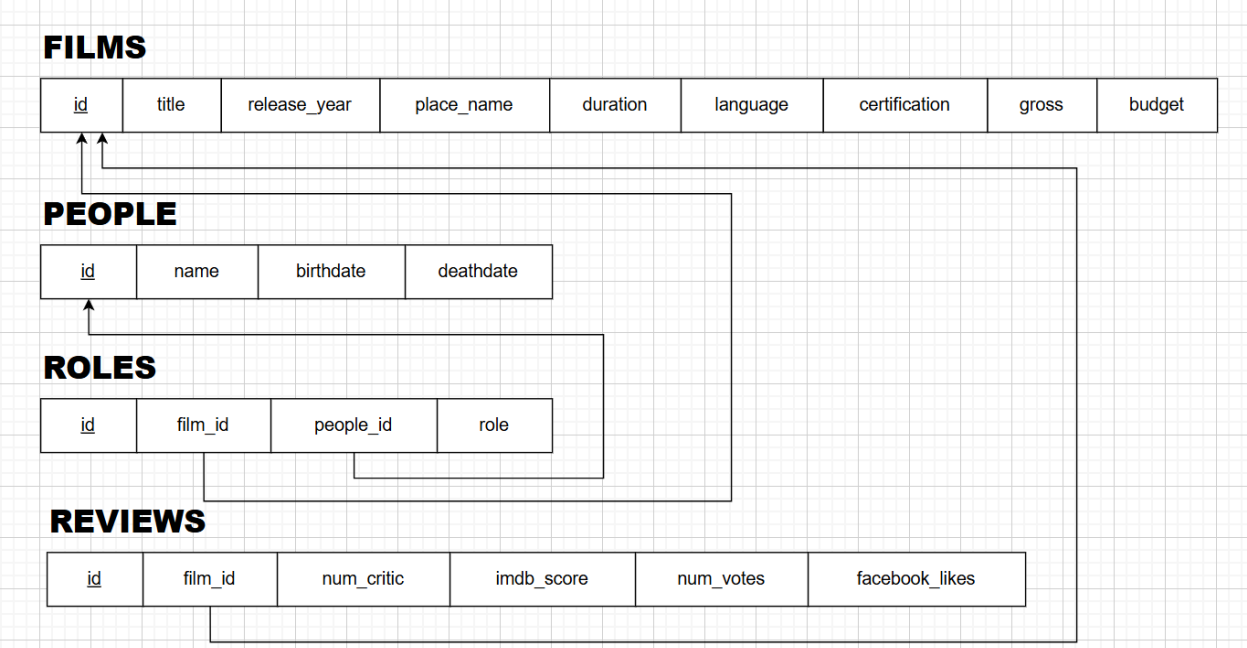


Figure: Logical Model / Design

Logical Model Explanation

In this logical model, the relationship between **FILMS** and **REVIEWS** is implemented by using film_id as a foreign key in the **REVIEWS** relation. This means that each review record is linked to one film.

The relationship between **FILMS** and **ROLES** is implemented by also using the film_id as a foreign key in the **ROLES** relation. The relationship between **PEOPLE** and **ROLES** is implemented by using people_id as a foreign key in the **ROLES** relation.

The **ROLES** relation is used as an associative relation to resolve the many-to-many relationship between **FILMS** and **PEOPLE**. A film can involve many people, and a person can be involved in many films. Thus, this many-to-many relationship is converted into two one-to-many relationships: **FILMS** to **ROLES**, and **PEOPLE** to **ROLES**.

Normalization Evaluation:

The current model structure are already satisfied **First Normal Form (1NF)**, **Second Normal Form (2NF)**, and **Third Normal Form (3NF)**.

The relations satisfy **1NF** because all attributes contain atomic values, and each relation has a primary key to uniquely identify each record. There are no repeating groups or multi-valued attributes stored in a single field.

The relations satisfy **2NF** because each relation uses a single-column primary key. So there is no partial dependency on part of a composite key. For example, in **FILMS**, attributes such as title, release_year, duration, gross, and budget all depend on the primary key id.

The relations also satisfy **3NF** because non-key attributes depend on the primary key instead of other non-key attributes. For example, in **PEOPLE**, name, birthdate, and deathdate depend on the person id. In **REVIEWS**, num_critic, imdb_score, num_votes, and facebook_likes describe the review record identified by id.

No mandatory decomposition is required because the four relations already satisfy the main requirements of 1NF, 2NF, and 3NF. The design also avoids storing a direct many-to-many relationship between **FILMS** and **PEOPLE** by using **ROLES** as an associative relation.

QUESTION 3

File Name	Excel Sheet Row No	Issue Identified	How you solve?	Comments
people.csv	All	**birthdate** used non-standard date format such as 7/6/1975, which caused PostgreSQL DATE import error.	Converted date values to ISO format YYYY-MM-DD.	ISO date format is more stable and avoids DateStyle-dependent import errors.
people.csv	62	**deathdate** was earlier than **birthdate**.	Set the **deathdate** and **birthdate** to NULL	This violated the chk_life_dates constraint, which requires deathdate to be later than or equal to birthdate.
people.csv	247	**deathdate** was earlier than **birthdate**.	Set the **deathdate** and **birthdate** to NULL	This violated the chk_life_dates constraint, which requires deathdate to be later than or equal to birthdate.
people.csv	667	**deathdate** was earlier than **birthdate**.	Set the **deathdate** and **birthdate** to NULL	This violated the chk_life_dates constraint, which requires deathdate to be later than or equal to birthdate.
reviews.csv	56	Null values in attributes that should not allow nulls	Deleted the row	**film_id** is required as a foreign key which cannot be Null.
reviews.csv	(film_id=736)	**film_id** value is not linked to the id in films table.	Deleted the row	This violated the foreign key constraint fk_reviews_film. Each review must reference an existing film to maintain referential integrity
reviews.csv	(film_id=5060)	**film_id** value is not linked to the id in films table.	Deleted the row	This violated the foreign key constraint fk_reviews_film. Each review must reference an existing film to maintain referential integrity

File Name	Excel Sheet Row No	Issue Identified	How you solve?	Comments
roles.csv	(film_id=790)	**film_id** value is not linked to the id in films table.	Deleted the row	This violated the foreign key constraint fk_roles_film. Each role record must reference an existing film to maintain referential integrity.
roles.csv	(film_id=1369)	**film_id** value is not linked to the id in films table.	Deleted the row	This violated the foreign key constraint fk_roles_film. Each role record must reference an existing film to maintain referential integrity.

Cleaning Explanation:

For date-related problems in people.csv, invalid or inconsistent dates were cleaned before importing the file into PostgreSQL. Date values were converted into ISO format YYYY-MM-DD to avoid DateStyle-dependent errors. Rows where deathdate was earlier than birthdate were treated as invalid because they violated the check constraint on life dates. Since the correct dates could not be reliably determined, the problematic date values were set to NULL.

For reviews.csv and roles.csv, rows with missing or invalid foreign key values were removed. For example, some film_id values in reviews and roles did not exist in the films table. Keeping these rows would violate foreign key constraints and damage referential integrity. Therefore, these rows were deleted from the cleaned dataset before importing the data into the final tables.

Table Creation Code:

```sql
CREATE TABLE films (
    id INTEGER PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    release_year INTEGER,
    place_name VARCHAR(100),
    duration INTEGER,
    language VARCHAR(100),
    certification VARCHAR(50),
    gross BIGINT,
    budget BIGINT,
    CONSTRAINT chk_release_year CHECK (release_year IS NULL OR release_year BETWEEN 1880 AND 2030),
    CONSTRAINT chk_duration CHECK (duration IS NULL OR duration > 0),
    CONSTRAINT chk_gross CHECK (gross IS NULL OR gross >= 0),
    CONSTRAINT chk_budget CHECK (budget IS NULL OR budget >= 0)
);

CREATE TABLE people (
    id INTEGER PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    birthdate DATE,
    deathdate DATE,
    CONSTRAINT chk_life_dates CHECK (deathdate IS NULL OR birthdate IS NULL OR deathdate >= birthdate)
);

CREATE TABLE reviews (
    id INTEGER PRIMARY KEY,
    film_id INTEGER NOT NULL,
    num_critic INTEGER,
    imdb_score NUMERIC(3,1),
    num_votes INTEGER,
    facebook_likes INTEGER,
    CONSTRAINT fk_reviews_film
        FOREIGN KEY (film_id) REFERENCES films(id),
    CONSTRAINT chk_imdb_score CHECK (imdb_score IS NULL OR imdb_score BETWEEN 0 AND 10),
    CONSTRAINT chk_num_critic CHECK (num_critic IS NULL OR num_critic >= 0),
    CONSTRAINT chk_num_votes CHECK (num_votes IS NULL OR num_votes >= 0),
    CONSTRAINT chk_facebook_likes CHECK (facebook_likes IS NULL OR facebook_likes >= 0)
);

CREATE TABLE roles (
    id INTEGER PRIMARY KEY,
    film_id INTEGER NOT NULL,
    people_id INTEGER NOT NULL,
    role VARCHAR(100),
    CONSTRAINT fk_roles_film
        FOREIGN KEY (film_id) REFERENCES films(id),
    CONSTRAINT fk_roles_people
        FOREIGN KEY (people_id) REFERENCES people(id)
);
```

The data was imported in the order of films, people, reviews, and roles. Importing parent tables first avoids foreign key constraint errors.

films table:

Query | Query History

```
1  SELECT * FROM films;
```

Data Output | Messages | Notifications

Showing rows: 1 to 1000 | Page

	id [PK] integer	title character varying (255)	release_year integer	place_name character varying (100)	duration integer	language character varying (100)	certification character varying (50)	gross bigint	budget bigint
1	1	Intolerance: Love's ...	1916	USA	123	[null]	Not Rated	[null]	385907
2	2	Over the Hill to the ...	1920	USA	110	[null]	[null]	300000(	100000
3	3	The Big Parade	1925	USA	151	[null]	Not Rated	[null]	245000
4	4	Metropolis	1927	Germany	145	German	Not Rated	26435	6000000
5	5	Pandora's Box	1929	Germany	110	German	Not Rated	9950	[null]
6	6	The Broadway Melo...	1929	USA	100	English	Passed	280800(	379000
7	7	Hell's Angels	1930	USA	96	English	Passed	[null]	3950000
8	8	A Farewell to Arms	1932	USA	79	English	Unrated	[null]	800000
9	9	42nd Street	1933	USA	89	English	Unrated	230000(	439000
10	10	She Done Him Wrong	1933	USA	66	English	Approved	[null]	200000
11	11	It Happened One Ni...	1934	USA	65	English	Unrated	[null]	325000
12	12	Top Hat	1935	USA	81	English	Approved	300000(	609000

	id	title	release_ye	place_nam	duration	language	certificatio	gross	budget
1									
2	1	Intolerance	1916	USA	123		Not Rated		385907
3	2	Over the H	1920	USA	110			3000000	100000
4	3	The Big Pa	1925	USA	151		Not Rated		245000
5	4	Metropolis	1927	Germany	145	German	Not Rated	26435	6000000
6	5	Pandora's	1929	Germany	110	German	Not Rated	9950	
7	6	The Broad	1929	USA	100	English	Passed	2808000	379000
8	7	Hell's Ange	1930	USA	96	English	Passed		3950000
9	8	A Farewell	1932	USA	79	English	Unrated		800000
10	9	42nd Stree	1933	USA	89	English	Unrated	2300000	439000
11	10	She Done	1933	USA	66	English	Approved		200000
12	11	It Happene	1934	USA	65	English	Unrated		325000

The output of SELECT * FROM films shows that the cleaned film data was successfully imported into the films table. The table contains the expected columns, including id, title, release_year, place_name, duration, language, certification, gross, and budget. The id column is used as the primary key, and the imported records match the cleaned CSV file.

people table:

```
2  SELECT * FROM people;
```

Data Output | Messages | Notifications

	id [PK] integer	name character varying (255)	birthdate date	deathdate date
1	1	50 Cent	1975-07-06	[null]
2	2	A. Michael Baldwin	1963-04-04	[null]
3	3	A. Raven Cruz	[null]	[null]
4	4	A.J. Buckley	1978-02-09	[null]
5	5	A.J. DeLucia	[null]	[null]
6	6	A.J. Langer	1974-05-22	[null]
7	7	Aaliyah	1979-01-16	2001-08-25
8	8	Aaron Ashmore	1979-10-07	[null]
9	9	Aaron Hann	[null]	[null]
10	10	Aaron Hill	1983-04-23	[null]

	A	B	C	D
1	id	name	birthdate	deathdate
2	1	50 Cent	1975/7/6	
3	2	A. Michael	1963/4/4	
4	3	A. Raven Cruz		
5	4	A.J. Buckley	1978/2/9	
6	5	A.J. DeLucia		
7	6	A.J. Langer	1974/5/22	
8	7	Aaliyah	1979/1/16	2001/8/25
9	8	Aaron Ash	1979/10/7	
10	9	Aaron Hann		

The output of SELECT * FROM people confirms that the cleaned people data was successfully imported. The table includes id, name, birthdate, and deathdate. The date values were converted into the standard YYYY-MM-DD format before import. Some invalid or inconsistent date values(such as: **deathdate** was earlier than **birthdate**) were set to NULL to satisfy the chk_life_dates constraint.

reviews table:

```
3  SELECT * FROM reviews;
```

Data Output | Messages | Notifications

	id [PK] integer	film_id integer	num_critic integer	imdb_score numeric (3,1)	num_votes integer	facebook_likes integer
1	3934	588	432	7.1	203461	46000
2	3405	285	267	6.4	149998	0
3	478	65	29	3.2	8465	491
4	74	83	25	7.6	7071	930
5	1254	1437	224	8.0	241030	13000
6	740	111	64	6.4	64742	0
7	4841	1058	579	8.1	479047	117000
8	2869	59	104	6.8	18442	689
9	3252	117	160	7.2	49855	10000
10	1181	163	99	7.3	16995	0

	A	B	C	D	E	F
1	id	film_id	num_critic	imdb_sco	num_votes	facebook_likes
2	3934	588	432	7.1	203461	46000
3	3405	285	267	6.4	149998	0
4	478	65	29	3.2	8465	491
5	74	83	25	7.6	7071	930
6	1254	1437	224	8	241030	13000
7	740	111	64	6.4	64742	0
8	4841	1058	579	8.1	479047	117000
9	2869	59	104	6.8	18442	689
10	3252	117	160	7.2	49855	10000

The output of SELECT * FROM reviews shows that the cleaned review data was imported correctly. Each review record has a valid film_id that references an existing film in the films table. Rows with missing film_id values or film_id values not found in the films table were deleted before import to maintain referential integrity.

roles table:

```
4   SELECT * FROM roles;
```

Data Output | Messages | Notifications

	id [PK] integer	film_id integer	people_id integer	role character varying (100)
1	1	1	1630	director
2	2	1	4843	actor
3	3	1	5050	actor
4	4	1	8175	actor
5	5	2	3000	director
6	6	2	4019	actor
7	7	2	5274	actor
8	8	2	7449	actor
9	9	3	1459	actor
10	10	3	3929	actor

	A	B	C	D
1	id	film_id	people_id	role
2	1	1	1630	director
3	2	1	4843	actor
4	3	1	5050	actor
5	4	1	8175	actor
6	5	2	3000	director
7	6	2	4019	actor
8	7	2	5274	actor
9	8	2	7449	actor
10	9	3	1459	actor

The output of SELECT * FROM roles confirms that the cleaned role data was successfully imported. Each role record is linked to an existing film through film_id and an existing person through people_id. Rows with invalid foreign key values were removed before import to avoid violating the foreign key constraints.

Summary:

I assumed that each table's `id` column is the primary key, while `film_id` and `people_id` are foreign keys used to connect the tables. During data cleaning, rows with missing or invalid foreign key values were removed because they could not be linked to valid parent records. Date values were converted to ISO format `YYYY-MM-DD` for PostgreSQL compatibility. If a date was inconsistent, such as `deathdate` being earlier than `birthdate`, the unreliable date values were set to NULL instead of being guessed. This approach helped maintain data integrity and avoided fabricating data.

Some approaches were used to support the data cleaning process. LEFT JOIN queries were used to find foreign key values in reviews and roles which did not match the primary keys in films and people. Excel filtering was used to identify inconsistent date values, such as rows where deathdate was earlier than birthdate. Invalid foreign key rows were deleted, while unreliable date values were set to NULL instead of being guessed.

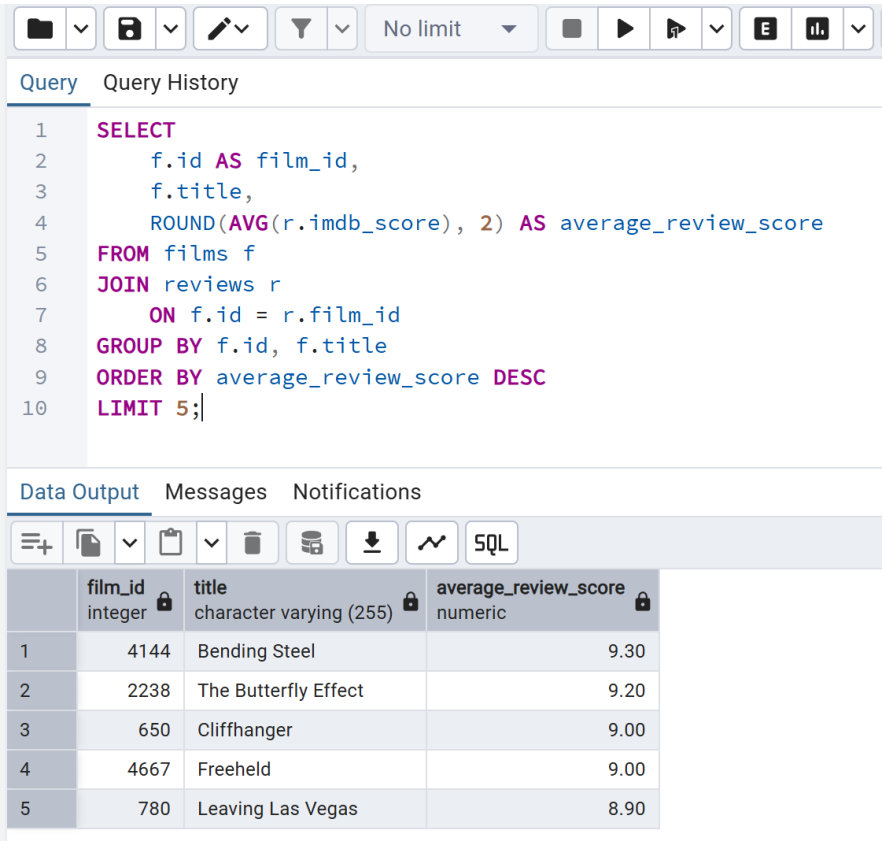
Xi'an Jiaotong-Liverpool University
西交利物浦大学

QUESTION 4

Query 1: Top 5 films by average review score:

Code:

```
SELECT
    f.id AS film_id,
    f.title,
    ROUND(AVG(r.imdb_score), 2) AS average_review_score
FROM films f
JOIN reviews r
    ON f.id = r.film_id
GROUP BY f.id, f.title
ORDER BY average_review_score DESC
LIMIT 5;
```



	film_id integer	title character varying (255)	average_review_score numeric
1	4144	Bending Steel	9.30
2	2238	The Butterfly Effect	9.20
3	650	Cliffhanger	9.00
4	4667	Freeheld	9.00
5	780	Leaving Las Vegas	8.90

This query joins the films table with the reviews table using films.id = reviews.film_id. It then groups the records by film id and film title, calculates the average IMDb score for each film using AVG(), and orders the results from highest to lowest average score. LIMIT 5 is used to display only the top five films with the highest average review score.

Relational Algebra:

Top5(
  τ avg_score DESC (
    γ f.id, f.title; AVG(r.imdb_score) → avg_score
    (films ⋈ films.id = reviews.film_id reviews)
)
)

The join operation connects each film with its review records. The grouping operation groups records by film id and title, and calculates the average IMDb score for each group. The sorting operation orders the films by average score in descending order, and Top5 returns the first five records.

Query 2: Films that have received more than 5 reviews:

Code:

```
SELECT
    f.id AS film_id,
    f.title,
    COUNT(r.id) AS number_of_reviews
FROM films f
JOIN reviews r
    ON f.id = r.film_id
GROUP BY f.id, f.title
HAVING COUNT(r.id) > 5
ORDER BY number_of_reviews DESC;
```

No limit

Query | Query History

```
17  SELECT
18      f.id AS film_id,
19      f.title,
20      COUNT(r.id) AS number_of_reviews
21  FROM films f
22  JOIN reviews r
23      ON f.id = r.film_id
24  GROUP BY f.id, f.title
25  HAVING COUNT(r.id) > 5
26  ORDER BY number_of_reviews DESC;
```

Data Output | Messages | Notifications

Showing rov

	film_id integer	title character varying (255)	number_of_reviews bigint
1	1	Intolerance: Love's Struggle Throughout the Ages	51
2	2	Over the Hill to the Poorhouse	32
3	26	The Blue Bird	32
4	3	The Big Parade	32

This query joins films with reviews using the film_id foreign key. It groups the data by film id and title, then uses COUNT() to calculate the number of review records for each film. The HAVING clause filters the grouped results and only keeps films with more than five reviews. The results are ordered by the number of reviews in descending order.

Relational Algebra:

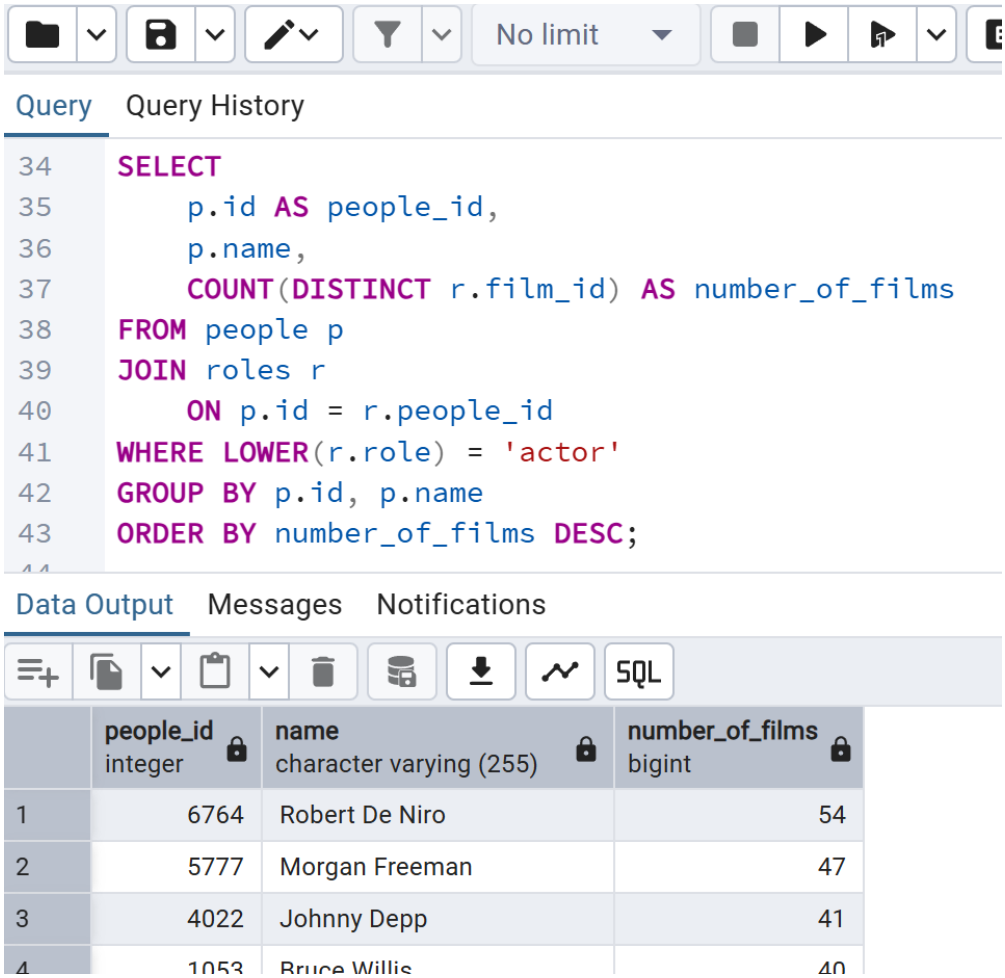
τ review_count DESC (
  σ review_count > 5 (
    γ f.id, f.title; COUNT(r.id) $\rightarrow$ review_count
    (films $\bowtie$ films.id = reviews.film_id reviews)
)
)

The join links films with their review records. The grouping operation groups by film id and film title and counts the number of reviews for each film. The selection operation keeps only the films where the review count is greater than five.

Query 3: Number of films each actor has appeared in:

Code:

```
SELECT
    p.id AS people_id,
    p.name,
    COUNT(DISTINCT r.film_id) AS number_of_films
FROM people p
JOIN roles r
    ON p.id = r.people_id
WHERE LOWER(r.role) = 'actor'
GROUP BY p.id, p.name
ORDER BY number_of_films DESC;
```



	people_id integer	name character varying (255)	number_of_films bigint
1	6764	Robert De Niro	54
2	5777	Morgan Freeman	47
3	4022	Johnny Depp	41
4	1053	Bruce Willis	40

This query joins the people table with the roles table using people.id = roles.people_id. Since the roles table contains both director and actor values, the WHERE clause is used to keep only records where role is actor. The query then groups the results by person id and name, and COUNT(DISTINCT r.film_id) calculates how many different films each actor has appeared in.

Xi'an Jiaotong-Liverpool University
西交利物浦大學

QUESTION 5

Reflection on Database Design:

The topic that helped me most in this course was database normalization, especially the Week 6 lecture on 1NF, 2NF, 3NF and data anomalies. Before this topic, I mainly thought database design was just about creating tables and choosing suitable data types. However, the lecture helped me understand that a good database design should also reduce redundancy and prevent insertion, update, and deletion anomalies.

A useful class activity was the Kahoot game in the normalization lecture. The teacher divided us randomly into three teams. Each student answered questions about database normalization, and each correct answer allowed our team to build one more layer on our team's building. And finally the team that had the tallest building won. This activity made the topic more memorable because I had to quickly judge concepts such as whether a table was in 1NF or whether there were partial dependencies, and why 3NF helps remove transitive dependencies.

This knowledge was useful in Coursework 2. When designing the film database in question1 and question2, instead of storing repeated people information inside the films table I used a separate **roles** table to connect **films** and **people**. This reduced duplication and helped

represent the many-to-many relationship properly. The most challenging part for me was deciding whether the current tables already satisfied 3NF, but the Week 6 examples helped me check dependencies more systematically.

QUESTION 6

Reflection on Understanding the Limitations of Relational Database Design:

Through Coursework 1 and Coursework 2, I realized that relational databases are very useful for **structured data**, but they also have some limitations. In Coursework 2, the film database worked well because the data could be organized into clear tables such as **films**, **people**, **reviews**, and **roles**. Primary keys and foreign keys helped maintain data integrity, especially when checking whether film id and people id correctly matched existing records.

However, I also found that relational databases are quite strict. Before importing the data into PostgreSQL, the CSV files had to be **cleaned** carefully. For example, date values had to follow a valid format, foreign keys had to match primary keys, and incorrect rows could violate constraints. This showed me that relational databases require well-prepared and consistent data.

Another limitation is **flexibility**. If the dataset contains unstructured or semi-structured data, such as film posters, user comments, images, social media content, or flexible metadata, forcing everything into fixed tables may become difficult. Also, highly normalized designs may require many JOIN operations, which can make queries more complex.

For large-volume or diverse-format data, a different approach such as a **NoSQL document database** may be more suitable. For example, MongoDB could store flexible film metadata or user-generated comments more naturally. Overall, relational databases are strong for accuracy and consistency, but less flexible when handling large, varied, or unstructured data.

Xi'an Jiaotong-Liverpool University
西交利物浦大學

QUESTION 7

Efforts to Follow Coursework Instructions

I completed the coursework according to the order of the questions. Each question was answered in its own section, including ER diagram design, logical model and normalization, database construction and data import, SQL queries, relational algebra, and reflection questions.

During the coursework, I tried to keep the report clear and easy to follow. Diagrams, tables, SQL code, screenshots, and explanations were included where required. For Q3, I documented the data cleaning process, listed the issues identified in the dataset, and provided verification screenshots after importing the cleaned data into PostgreSQL. For Q4, I included SQL queries, output screenshots, brief explanations, and relational algebra expressions where required.

I also checked that the database name, table names, primary keys, foreign keys, and constraints were consistent throughout the report and SQL files. The cleaned CSV files and SQL code files were prepared for submission so that the database can be replicated and verified.

Tools and Technologies Used:

- **PostgreSQL**: used to create the film_db database, define tables, apply constraints, and import data.
- **pgAdmin**: used to execute SQL scripts, import CSV files, and capture query output screenshots.
- **Microsoft Excel**: used to inspect and clean CSV data, especially date formatting and invalid values.
- **draw.io**: used to create the ER diagram and logical model diagrams.
- **SQL**: used for table creation, data validation, data import verification, joins, grouping, aggregation, and relational database operations.
- **CSV files**: cleaned and used as the final data source for importing into PostgreSQL.


Xi'an Jiaotong-Liverpool University
西交利物浦大學